SudhaarAI — Code Generation and Context-Based Code Optimizer

Group Members: Raqib Hayat, Abu Bakar **Supervisor:** Shahzad Arif **Co-Supervisor:** Dr. Shafiq ur Rahman

1. Executive Summary

SudhaarAI is an AI-powered Python code optimization platform combining static analysis, dynamic profiling, semantic understanding (CodeBERT), and Retrieval-Augmented Generation (RAG).

- **Frontend:** Next.js 16 + React 19 (Tailwind 4, Zustand)
- **Backend:** Python Flask REST API
- **Vector DB:** ChromaDB (Semantics/RAG)
- **Doc DB:** MongoDB Atlas (Sessions, Projects)
- **Auth:** Clerk (JWT-based RBAC)

2. Technology Stack Overview

Frontend Stack

Framework: Next.js 16 (App Router, SSR, routing)
UI Library: React 19
Language: TypeScript (Strict mode)
Styling: Tailwind CSS 4
Components: Radix UI + Shadcn (34 primitives)
State: Zustand (6 domain-specific stores)
Auth: Clerk (Authentication + middleware)
Animations: Framer Motion
Fonts: Syne, Space Grotesk, JetBrains Mono
Notifications: Sonner

Backend Stack

Runtime: Python 3.11+
Web Framework: Flask
Package Manager: uv
Vector DB: ChromaDB
Document DB: MongoDB (pymongo)
LLM SDKs: groq, openai, anthropic
ML Models: CodeBERT (HuggingFace)
Embeddings: sentence-transformers (768-dim)
Auth: PyJWT + Clerk JWKS

3. Frontend Architecture

State Management (Zustand)

Domain-driven store architecture:

- `chatStore`: Sessions, messages, model selection.
- `optimizeStore`: 8-step pipeline state machine.
- `projectStore`: Project CRUD.
- `notificationStore`: Toast/notification queue.
- `settingsStore`: User preferences.
- `sidebarStore`: Sidebar UI state.

Facade Hooks: `useChat()` and `useOptimize()` compose multiple stores and API calls into singular components. For example, `useOptimize` reads from `chatStore` for context and `projectStore` for scope.

Optimization State Machine

The `useOptimize` hook drives a 3-phase interactive pipeline via `optimizeStore`:

1. `idle`: Standby state
2. `analyzing`: Running static/dynamic analysis
3. `generating_query`: Preparing retrieval prompt
4. `query_ready`: Retrieval parameters computed
5. `retrieving`: Executing ChromaDB search
6. `practices_ready`: Extracted best practices
7. `optimizing`: Applying code refactoring
8. `done`: Refactoring verified and applied

4. Backend Architecture

Synchronous Python Flask 3.11+ application. Communication with frontend is strict HTTP/JSON with standard bearer tokens (no WebSockets/SSE).

5. Code Analysis Features

The Feature Vector $X = [F_s, F_d, F_m, F_b]$.

Static (F_s) & Dynamic (F_d)

- **Static (AST based):** `loc`, `cyclomatic_complexity`, `max_loop_depth`, `fan_in_out`, `recursive_functions`, `dead_code_ratio`, `redundant_computations`, `index_based_iteration`.
- **Dynamic (Profiler):** `exec_time`, `peak_memory`, `cpu_utilization`, `io_wait_time`, `gc_collections`.

Semantic (F_m) & Violations (F_b)

- **Semantic:** CodeBERT embedding (768-dim), computes similarity to optimal patterns $\cos(F_m, F_{opt})$.
- **Violations:**
  - `bp1_manual_loops`: Iteration instead of builtins
  - `bp2_io_in_loops`: I/O ops inside loops
  - `bp3_missing_caching`: Repeated computations
  - `bp4_unvectorized`: Missing NumPy vectorization
  - `bp5_anti_patterns`: General detection
  - Severity weighting applies per violation

6. 8-Step Core Pipeline

Phase 1: Analysis

1. **Feature Extraction**: Collect F_s, F_d, F_m, F_b via AST, runtime profiling, CodeBERT, and regex pattern matching.
2. **Normalize**: Normalize features against domain baseline X.
3. **Compute Score**: Calculates weighted sum.

$$S(C) = \sum w_i \hat{f}_i + w_s \cos(F_m, F_{opt})$$

Weights: Violations (60% - Dominant), Static (25%), Dynamic (15%), Semantic (0.3 additive bonus).

4. **Threshold**: Optimized if $S(C) \geq \tau_p$ (threshold defined by project-type).

Phase 2: Optimization (RAG)

5. **Retrieve**: Generate specialized query from violations → User reviews/edits → Retrieve best practices from ChromaDB + Project KB.
6. **Refactor**: Build optimization prompt merging code, practices, and violations → LLM generates optimized code C'.
7. **Validate**: Recompute score $S(C')$. Execute differential correctness test requiring semantic similarity $\cos(F_m, F'_m) \geq 0.95$ and sandboxed run output match.
8. **Output**: Return C', $S(C')$, and detailed verification metadata.

7. RAG Pipeline Subsystems

Query Processing

`QueryClassifier`: Directs intent (`code_search`, `doc_search`, `optimization`, `general`).
`QueryRouter`: Selects ChromaDB collections, weights, and strategies (filtered, hybrid).
`QueryRewriter`: LLM enhancement to rewrite and expand the query conceptually for maximal recall.

Retrieval System

- **Collections**: `code_blocks`, `combined`, `concepts`, `project_kb_{id}`.
- **Strategy**: Hybrid search default (70% semantic + 30% keyword frequency).
- **Boost**: Project specific KB documents get localized 2.0x ranking boost to prefer repository-aware contexts.

Context Assembly

`DocumentReranker` rescores documents using a cross-encoder heuristic. `ContextBuilder` truncates content to a 2048 token budget ensuring diversity > 0.85. `CitationTracker` resolves chunk IDs to human-readable names and maps inline citations `[1]`, `[2]`.

8. LLM Providers Interface

Integrations follow `LLMProvider(ABC)` implementing `generate(prompt, system, temp)`, `chat(messages, temp)`, and `count_tokens(text)`. Standard format returns `response`, `usage`, `model`, and `finish_reason`.

Provider Matrix & Fallback Chain

`Groq (Llama3.3-70B/Qwen3)` → `OpenAI (GPT-4o-mini)` → `Anthropic (Claude 3.5)` → `Ollama (Mistral/Qwen Local)`.

9. Database Schema

MongoDB Models

- **SESSIONS**: `sessionId` (PK), `userId` (FK), `title`, `projectId`, `messages`, `type`.
- **PROJECTS**: `id` (PK), `userId` (FK), `name`, `description`, `createdAt`.
- **PROJECT_FILES**: `id` (PK), `projectId` (FK), `original_filename`, `kb_status` (indexed | pending | failed).

ChromaDB Collections

Powered by `sentence-transformers` (768-dim embeddings).

- `code_blocks`: Snippets from documentation.
- `combined`: Mixed instruction and code content.
- `concepts`: Broad conceptual explanations.
- `project_kb_{id}`: Per-project dynamic vectors.